

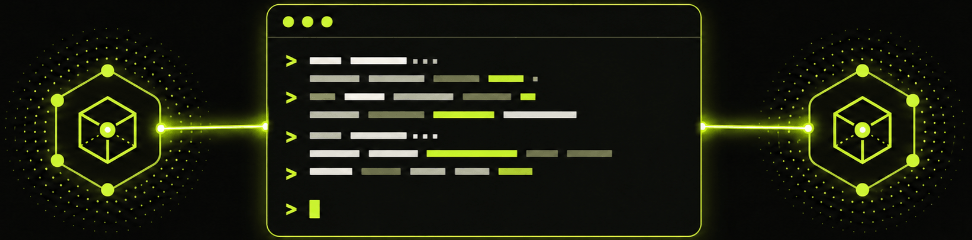
p2p-netcat

Netcat addressed by PeerId

One cryptographic address for terminal, TCP, UDP, and proxy access across NAT.

p2p-netcat web

A terminal between two peers.
Right in your browser.



IDENTITY

PeerId instead of IP

ROUTE

First authenticated route

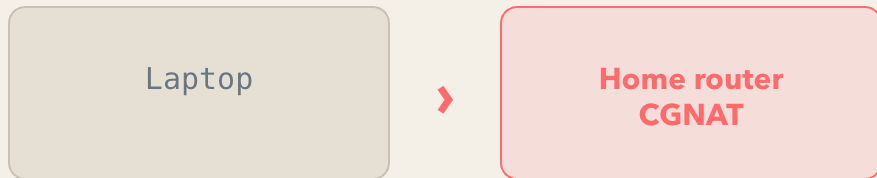
SESSION

TCP · UDP · PTY · SOCKS

Your home server has no public address – but you still need SSH.

WITHOUT P2P-NETCAT

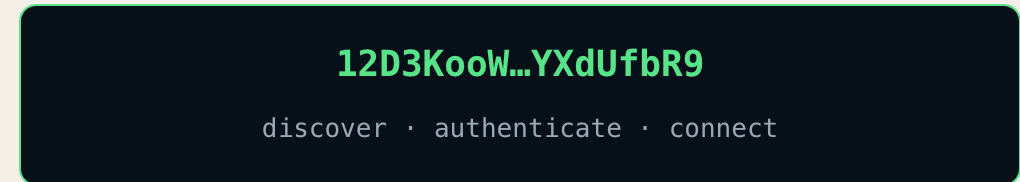
The network edge is not yours



Port forwarding is unavailable. Dynamic DNS does not help. A hosted VPN adds another control plane.

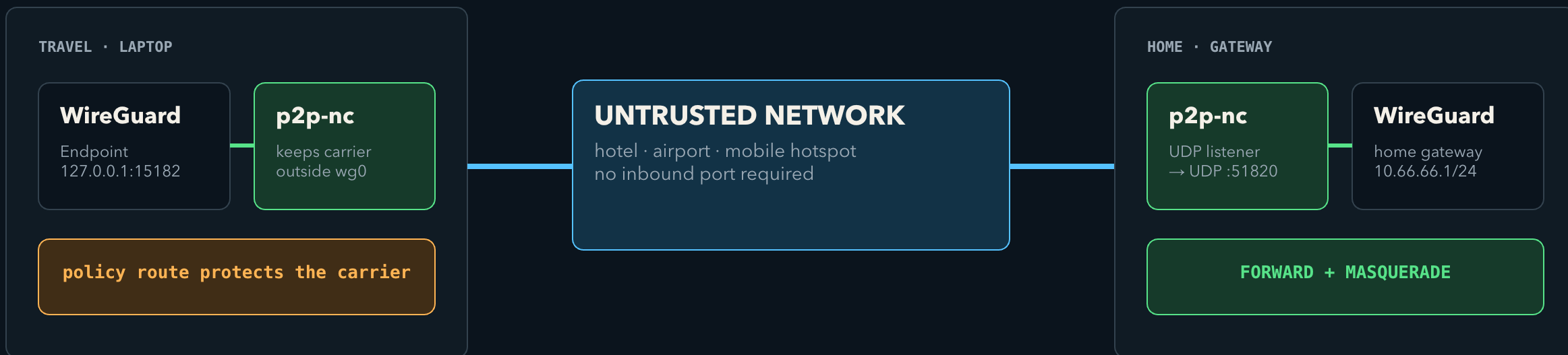
WITH P2P-NETCAT

The server keeps one identity



Run a listener at home. From anywhere, dial its PeerId and forward local port 2222 to SSH.

A full Internet tunnel should still work when the gateway is behind NAT.



ONE ROUTE

bring up carrier



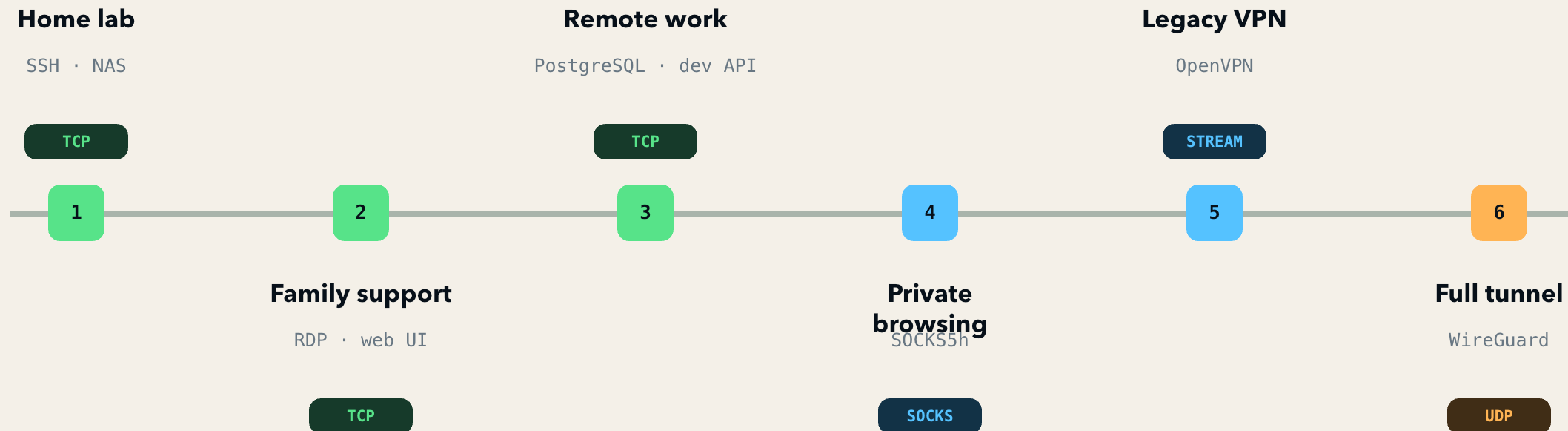
wg-quick up wg0



public IP = home

The P2P carrier reaches the home peer first; WireGuard then routes all Internet traffic through it.

The same need keeps repeating: reach a private service without owning its network edge.

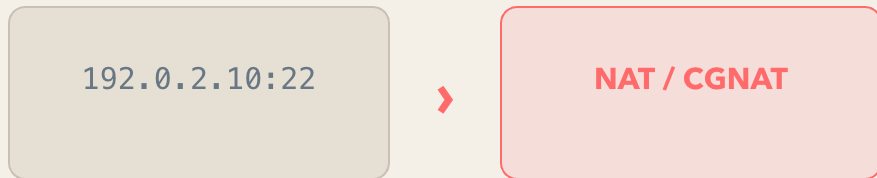


One stable PeerId replaces per-network IP addresses, port mappings, and temporary VPN control planes.

PeerId solves identity. The route still has to be found.

CLASSIC FORWARDING

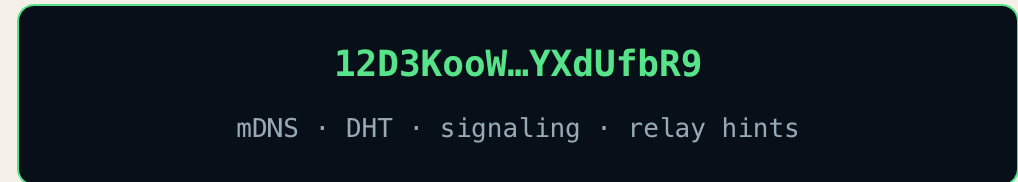
The IP changes – access breaks



DNS, port forwarding, a VPN concentrator, or manually operated infrastructure.

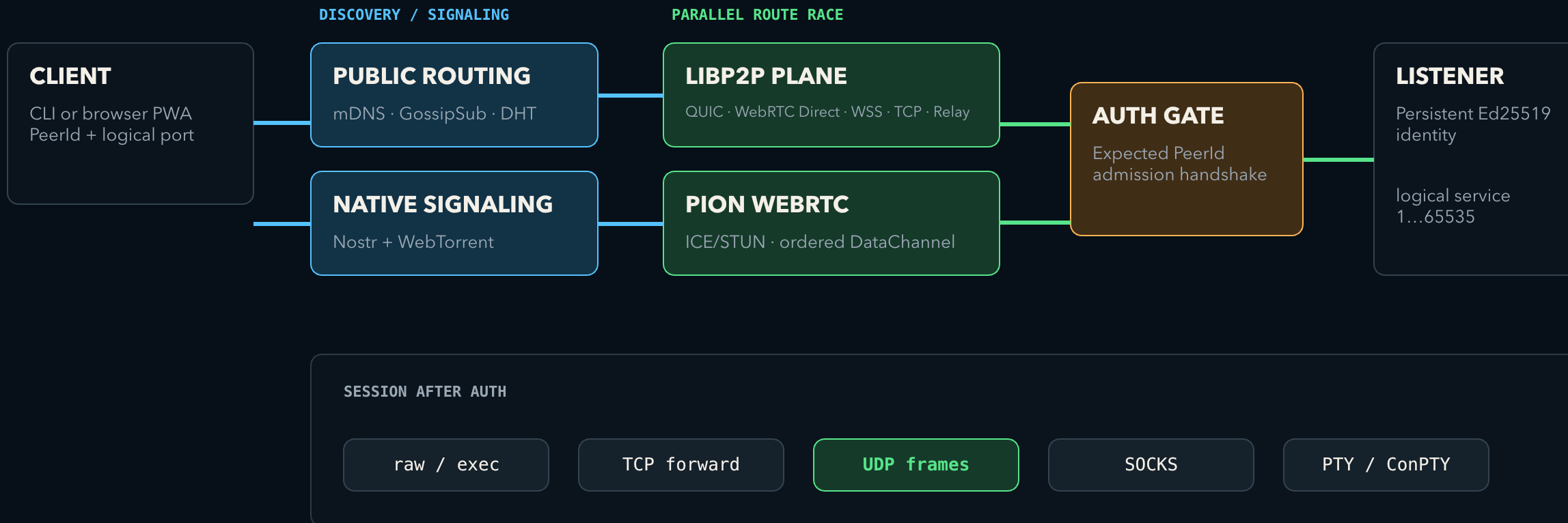
P2P-NETCAT

Identity stays stable



PeerId selects the right peer; discovery and route racing select a reachable path.

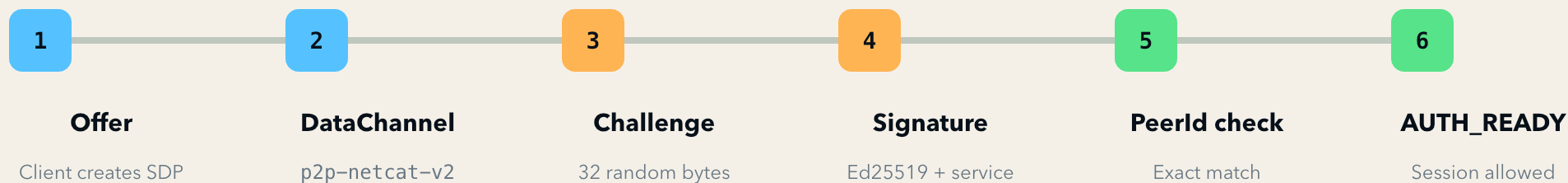
The first route to complete authentication wins.



One CLI covers terminals, streams, datagrams, and proxies.

MODE	LOCAL INTERFACE	REMOTE ACTION	WHAT IS PRESERVED	FLAG
RAW / EXEC	stdin/stdout	stream or command	byte stream	-e
TCP FWD	127.0.0.1:PORT	dial TCP target	connection	-p
UDP FWD	127.0.0.1:PORT/udp	connected UDP socket	packet boundaries	-u -p
SOCKS	SOCKS4/4a/5	CONNECT remotely	remote DNS	-S
PTY	interactive TTY	Unix PTY / ConPTY	resize + EOF	-i
LISTENER	logical port	multiple sessions	identity + policy	-l -k
Wire protocol compatibility				
	/p2p-netcat/1.0.0/<port> · /p2p-netcat/udp/1.0.0/<port>			

Application bytes flow only after the expected PeerId is proven.



An unverified WebRTC candidate cannot start the listener's PTY or application stream.

AFTER AUTH

**256
KiB**

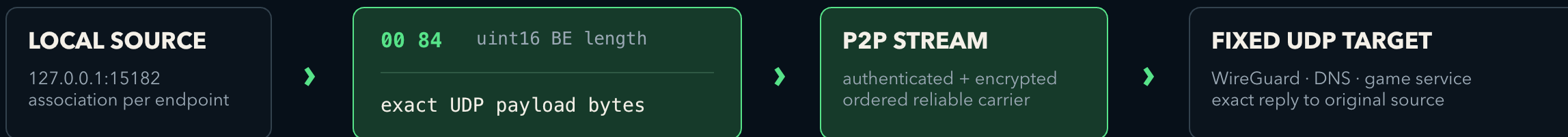
end-to-end flow
window

120 sec

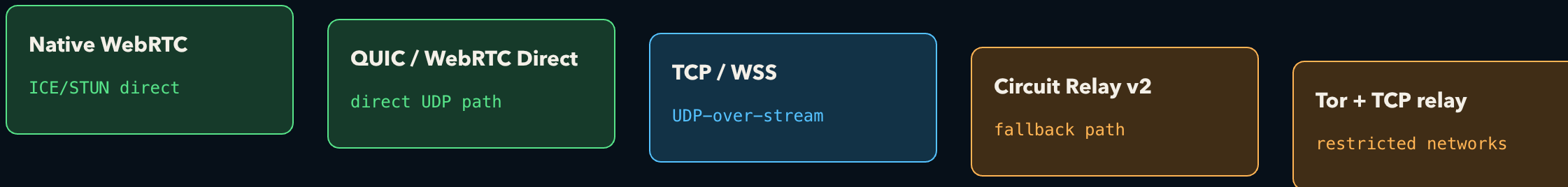
reconnect on the same
logical stream

UDP packet boundaries survive even over TCP/WSS/relay.

DATAGRAM ASSOCIATION

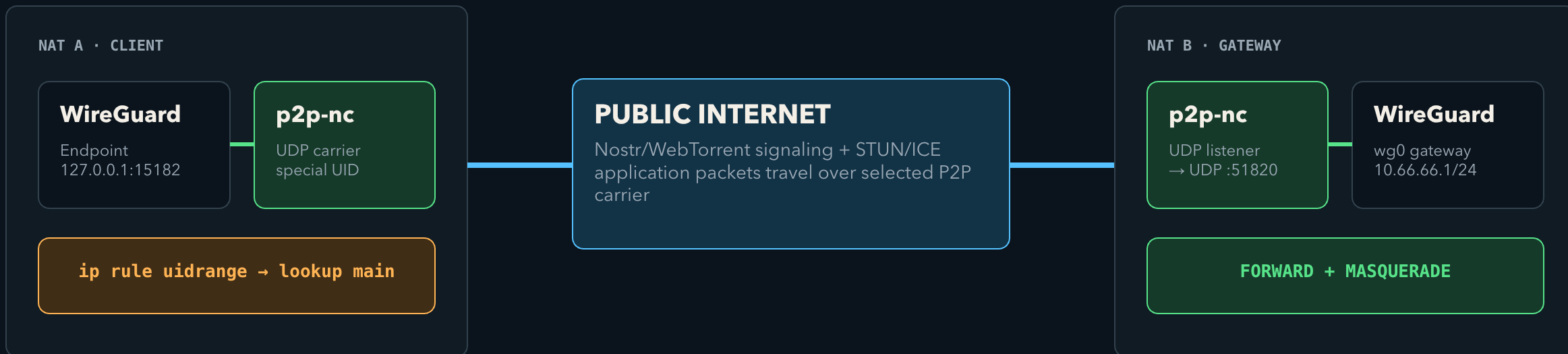


FALLBACK LADDER



Fallback cost: ordered delivery can introduce head-of-line blocking.

The carrier stays outside the default route, so the tunnel cannot consume itself.



E2E CHECK

two NAT namespaces



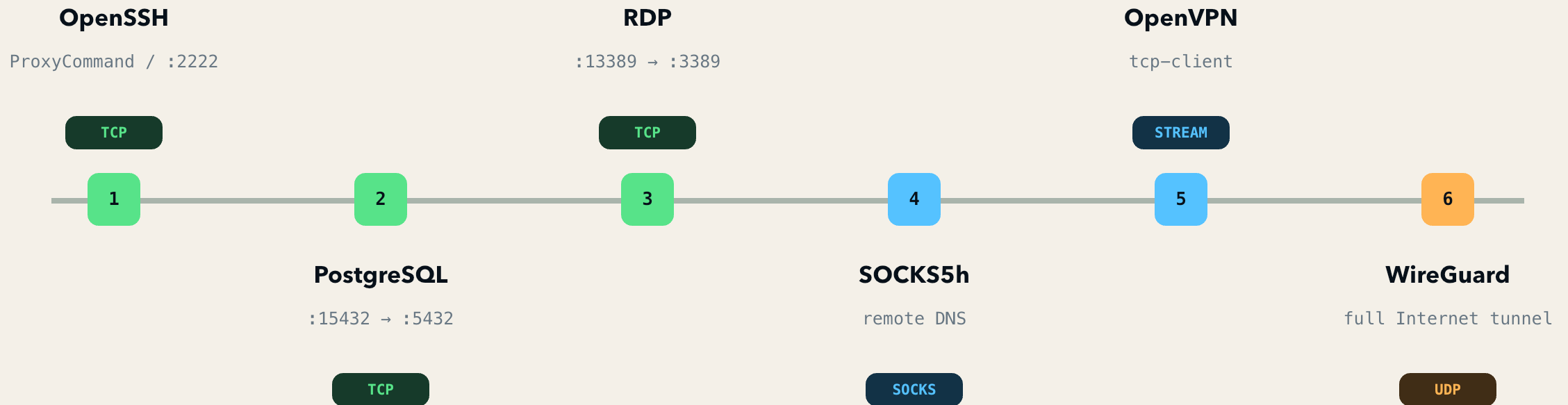
WireGuard handshake



HTTP egress = full-tunnel-ok

Start the carrier first, then wg-quick up wg0 – policy routing protects DNS, signaling, STUN, and reconnect.

One primitive operation powers many working tools.



The logical port selects the P2P service; it does not have to match the local TCP/UDP port.

One installation command; four delivery surfaces.

VERIFIED DEPLOY

```
curl -fsSL .../deploy.sh | bash
```

- detects OS + architecture
- downloads the release
- verifies SHA256SUMS
- installs p2p-nc + pnc

RELEASE BINARIES

Linux · macOS · Windows · Android
amd64 / arm64

GHCR CONTAINER

ghcr.io/santaklouse/go-p2p-netcat
non-root · /config identity

GO INSTALL

cmd/p2p-nc + cmd/pnc
CGO is not required

BROWSER PWA

Static

GitHub Pages

React UI
Web Worker
IndexedDB routes
Native WebRTC

Tests target the exact failure boundaries introduced by the architecture.

NATIVE WEBRTC SOAK

60 / 60

workload iterations

1 152 MiB

transferredBytes in a fresh JSON report

Pion DataChannel · SHA-256 · reconnect

CI COVERAGE

Go tests

Linux · macOS · Windows

Race detector

Linux · ./...

Full tunnel

2 NAT + WireGuard + HTTP

Docker

buildx + runtime checks

WebRTC soak

weekly Ubuntu + macOS

WHAT SOAK DOES NOT PROVE

- public Nostr/tracker availability
- intercontinental latency
- every NAT and firewall type
- background browser tabs and sleep
- 100% relay-free connectivity

Three command pairs cover the highest-value use cases.

Every example uses the documentation PeerId and real ports.

SSH THROUGH A LOCAL PORT

```
server$ p2p-nc -l -k -d 127.0.0.1 -p 22 22022
client$ p2p-nc -p 2222 12D3KooWQ3uxpHgjdKE6vGmvzKS8RPbxUDLwJ7XCLaD6YXdUfbR9 22022
client$ ssh -p 2222 alice@127.0.0.1
```

TCP PORT FORWARDING

```
server$ p2p-nc -l -k -d 127.0.0.1 -p 5432 25432
client$ p2p-nc -p 15432 12D3KooWQ3uxpHgjdKE6vGmvzKS8RPbxUDLwJ7XCLaD6YXdUfbR9 25432
```

WIREGUARD CARRIER FOR A FULL TUNNEL

```
server$ p2p-nc -u -l -k -d 127.0.0.1 -p 51820 35182
client$ sudo wireguard-full-tunnel.sh -- /usr/local/bin/p2p-nc -u --udp-idle-timeout 0 \
      -p 15182 12D3KooWQ3uxpHgjdKE6vGmvzKS8RPbxUDLwJ7XCLaD6YXdUfbR9 35182
client$ sudo wg-quick up wg0
```

Use p2p-netcat when identity should outlive the address.

PeerId is the stable entry point. Route racing and fallback are the delivery mechanism.

github.com/santaklouse/go-p2p-netcat

GOOD FIT

- access to home and edge hosts
- SSH and private TCP services
- UDP/WireGuard across NAT
- temporary tunnels without a control plane

PLAN B REQUIRED

- symmetric NAT or blocked UDP
- guaranteed enterprise ingress
- unsupported SOCKS UDP ASSOCIATE

TURN · Circuit Relay v2 · reachable TCP/WSS route